Figo Prefab Importer

User Manual & Setup Guide — Version 1.0.0 (Unity 2022.3+, Windows & macOS Editors)

Table of Contents

1. Overview

Figo Prefab Importer converts a **Figma design** (a `.fig` file saved with *File → Save local copy…*) or a **figo `canvas.json`** file into ready-to-use **UGUI prefabs** — `.prefab` files with baked textures, fonts and responsive anchors — entirely inside the Unity Editor. No account, no API key and no plugin inside Figma are required, and your design never leaves your machine.

Typical uses: bringing complete menu screens, HUDs and popups from Figma into Unity in one step, and keeping them in sync by simply re-converting after the design changes (all generated GUIDs are stable, so re-conversion produces clean version-control diffs).

2. Requirements

- Unity **2022.3 or newer**, Windows or macOS Editor.
- The `com.unity.ugui` package (present in any project that uses Unity UI).

- An internet connection *once*, for the first conversion, to download the small command-line converter (about 3 MB on Windows, about 10 MB on macOS). Fully offline installation is also possible — see section 8.

3. Installation & Setup

1. Import the package from the Asset Store (*Window → Package Manager → My Assets*) or via *Assets → Import Package*. All files are installed under `Assets/FigoPrefabImporter/`.
2. No further setup is required. The importer is an Editor extension; it adds:
   - the menu entry **Tools → Figo Prefab Importer...**, and
   - a context-menu entry **Figo → Convert to UGUI Prefabs** on `.fig` / `.json` assets in the Project window.
3. Optional: open the demo scene (section 4) to see a converted result immediately.

4. The Demo Scene

Open `Assets/FigoPrefabImporter/Samples/FigoDemo.unity`. The scene contains a Canvas (Screen Space – Overlay, reference resolution 1280×720) with two prefab instances that were generated by this importer from the bundled sample design `Samples/starfall_menu.canvas.json`:

- **menu** — the main-menu screen of the "Starfall" sample game UI (active by default);
- **settings** — the settings screen (inactive by default; disable *menu* and enable this instance in the Hierarchy to view it).

The prefabs themselves live in `Samples/StarfallPrefabs/` together with their baked textures. Running the tutorial in section 5 on the bundled sample reproduces exactly these prefabs, so you can compare your own conversion result against the shipped one.

5. Step-by-Step Tutorial: Converting Your First Design

1. Open **Tools → Figo Prefab Importer...** The importer window appears.
2. In **Design input**, click **Browse** and select a design file:
   - a `.fig` file saved from Figma via *File → Save local copy...*, or
   - a figo `canvas.json` (or Figma REST API JSON).
   - To follow along without your own file, pick the bundled sample: `Assets/FigoPrefabImporter/Samples/starfall_menu.canvas.json`.
3. In **Output**, enter a folder under `Assets/`, for example `Assets/FigoImported/MyMenu`. The folder is created if it does not exist.
4. Leave the **Options** at their defaults for now (they are explained in section 6).
5. Click **Convert**.

    - On the very first conversion the importer asks for permission to download the command-line converter for your platform (section 8). Confirm with **Download**; this happens only once per project.
6. When the progress bar finishes, the output folder is highlighted in the Project window. It contains one `.prefab` per top-level design frame plus a shared `textures/` folder.
7. Create a Canvas if your scene does not have one (*GameObject → UI → Canvas*), then drag any generated prefab onto the Canvas. The prefab keeps the design's pixel size and its responsive anchors. Done.

> **Tip:** you can also right-click a `.fig` or `.json` asset in the Project window and choose **Figo → Convert to UGUI Prefabs** — this opens the importer window with the input and a matching output folder already filled in.

6. Conversion Options Reference

Option	Meaning
Fonts folder (optional)	A folder containing the design's `.ttf` / `.otf` files. They are bundled as font assets and matched to each text element by family, weight and italic style. Without it, all text uses Unity's built-in font.
Frame filter (optional)	Convert only the top-level frame with this exact name. When empty, every top-level frame becomes one prefab.
Texture scale	Supersampling factor for baked sprites. `1` = design pixels, `2` = @2× textures for high-DPI targets.
Extract shared components	Detects repeated UI structures (cards, list rows, buttons) and emits them as nested prefabs in a `components/` subfolder, with per-instance overrides (requires Unity 2022.2 or newer).
Color space	**Auto Detect** reads the project's Player Settings; **Gamma** / **Linear** force one mode. See section 9 for details.

7. What the Importer Generates

- Each top-level design frame becomes one self-contained `.prefab` built from standard UGUI components only: `RectTransform`, `CanvasRenderer`, `UI.Image` and `UI.Text`. No custom runtime scripts are added, so the prefabs have no dependency on this package at run time.
- A shared `textures/` folder with deduplicated PNG sprites. Solid rectangles become sprite-less tinted `UI.Image` components (zero texture cost); gradients, vector shapes, images, strokes and effects are baked pixel-faithfully.
- Responsive constraints from the design are mapped to RectTransform anchors.

- All GUIDs and file IDs are derived from content: re-converting the same design produces identical files and therefore clean version-control diffs.

8. The Converter Binary (Download & Offline Installation)

Conversion is performed by `figo2unity`, a self-contained command-line program with no external dependencies. Your design file is processed locally; the network is used only to fetch the converter itself, once.

8.1 Automatic download (default)

On the first **Convert** the plugin offers to download the binary for your platform from the open-source figo repository and caches it in `Library/FigoPrefabImporter/` inside your project:

- Windows x64 (about 3 MB):
  `https://raw.githubusercontent.com/nowasm/figo/master/prebuild/win-x64/figo2unity.exe`
- macOS universal (Intel + Apple Silicon, macOS 11+, about 10 MB):
  `https://raw.githubusercontent.com/nowasm/figo/master/prebuild/macos/figo2unity`

If the `Library/` folder is ever deleted, the binary is simply downloaded again. To force a fresh copy, delete `Library/FigoPrefabImporter/` and convert again.

8.2 Offline / air-gapped machines

Download the file listed above on any machine and place it in the package's `Editor/Bin/` folder (create the folder next to `Editor/FigoPrefabImporter.cs`). A binary found there always takes precedence over the downloaded cache, and no network access is attempted.

9. Color Space: Gamma vs. Linear

Gamma output matches the design pixel for pixel. In a project that uses the **Linear** color space, UGUI blends translucent pixels in linear light, which makes glows, soft shadows and translucent panels render noticeably brighter than in the design tool. The importer's **Linear** mode pre-compensates the alpha of translucent pixels so the converted UI closely approximates the intended sRGB appearance.

The default **Auto Detect** mode reads your project's *Player Settings → Other Settings → Color Space* and picks the correct mode automatically; the window always shows which mode will be used. If you later switch the project's color space, simply re-convert the design.

10. Render Pipeline Compatibility (Built-in, URP, HDRP)

The importer is an Editor-only tool and the prefabs it generates are plain UGUI content — standard `Canvas` / `UI.Image` / `UI.Text` components rendered by Unity's UI system, which works identically under the **Built-in Render Pipeline**, the **Universal Render Pipeline (URP)** and the **High-Definition Render Pipeline (HDRP)**. Nothing in the package depends on Built-in-Render-Pipeline-only features (no lighting, no surface shaders, no post-processing hooks).

The package contains **no shaders and no materials** — the bundled sample prefabs and the demo scene use only Unity's default UI material. When you convert a design in a **Linear**-color-space project, the converter additionally generates a small unlit UI text shader (`Figo/UIText`, modeled on Unity's own *UI/Default*) into your output folder to keep text edges correct; like *UI/Default* it is render-pipeline independent and needs no conversion by the Render Pipeline Converter. Compatibility has been verified end to end on Unity 2022.3 with URP 14: the demo scene renders correctly through a URP camera.

No setup differences exist between pipelines — the tutorial in section 5 applies unchanged to Built-in, URP and HDRP projects.

11. Scripting Reference

No coding is required to use this asset. All functionality is available through the Editor window (*Tools → Figo Prefab Importer…*) and the Project-window context menu, and the generated prefabs consist of standard UGUI components only.

For completeness, the package contains a single Editor script, `Figo.PrefabImporter.FigoPrefabImporterWindow` (assembly `Figo.PrefabImporter.Editor`, namespace `Figo.PrefabImporter`), an `EditorWindow` that stages the input file, invokes the converter binary and refreshes the Asset Database. It is Editor-only and is never included in builds. There is no public runtime API.

12. Troubleshooting & FAQ

Symptom / question	Answer
"Conversion failed — see the Console for details."	The Console contains the converter's full log. The most common causes are an unsupported input file (the file must be a Figma `.fig`, a figo `canvas.json` or Figma REST JSON) and an output folder outside `Assets/`.
The download dialog fails or the machine has no internet access.	Fetch the binary manually and use the offline installation described in section 8.2.
Text looks different from the design.	Provide the design's font files via the **Fonts folder** option; without it, all text falls back to Unity's built-in font.

Glows or translucent panels look too bright.	Your project uses the Linear color space and the prefabs were exported in Gamma mode. Re-convert with **Color space** set to **Linear** (or **Auto Detect**). See section 9.
Does it work in URP or HDRP projects? Do I need the Render Pipeline Converter?	Yes, it works unchanged in Built-in, URP and HDRP projects, and no conversion step is needed — the output is plain UGUI rendered with Unity's default UI material. See section 10.
Does the asset add anything to my builds?	No. The package is Editor-only; the generated prefabs use only standard UGUI components and have no dependency on the package.
Is my design uploaded anywhere?	No. Conversion runs entirely on your machine. The only network access is the one-time download of the converter binary, and even that can be avoided (section 8.2).

13. Support

Questions, bug reports and feature requests: `https://github.com/nowasm/figo/issues`